

Gradient computation for the inverse problem: a comparison of differentiation methods

Sipan Bareyan

June 15, 2026

Abstract

We compare methods for computing the gradient $g = \nabla_V L$ of the misfit $L(V) = \int_0^{2\pi} (u(V) - u_0)^2 dx$, where the state $u(V)$ solves the discretized nonlinear equation $F(u, V) = 0$ by Newton’s method. The state solve and the adjoint construction are taken as given; we study only the *compute cost*, *memory*, and *accuracy* of the gradient, and how each scales with the number of parameters N . We contrast finite differences, automatic differentiation of the loss (forward and reverse), the hand-written adjoint, and AD equipped with a rule for the state solve.

1 Methods

The gradient maps the N entries of the discretized potential V to a scalar, so the question that matters is how its cost grows with N .

- **Finite differences (FD).** Perturb each entry in turn: $N+1$ state solves, with accuracy limited to $\sqrt{\varepsilon_{\text{mach}}}$ by the step-size trade-off. Baseline.
- **Forward-mode AD.** Carries one tangent direction per pass, so N passes (one per parameter). It does *not* unroll the solver to scalars: it propagates a dual number through the Newton iteration, so a pass costs a constant factor times one state solve. Machine-accurate, but with the same $O(N)$ scaling as FD.
- **Reverse-mode AD.** A single backward pass returns the whole gradient. It traces the Newton iteration and stores its intermediate states for the backward sweep, so memory grows with the iteration count.
- **Adjoint (analytic).** One extra linear solve with the operator of a Newton step, at the converged solution. Cost is independent of N and comparable to a single state solve. This is the reference.
- **Rule-based AD.** A custom rule for the state solve makes the AD tool compose the analytic adjoint instead of differentiating the iteration. It runs on the production solver and stays differentiable inside a larger pipeline.

The CG rule. Every AD path uses a custom rule for the inner conjugate-gradient solve, so the *linear* system is never differentiated through. What AD sees is only the *Newton* iteration. The naive-versus-ruled comparison therefore measures exactly the cost of unrolling Newton against the cost of replacing it by a rule, rather than the (well-known, easily avoided) cost of unrolling CG.

Table 1: Cost of one gradient evaluation. $T_{\text{sol}} = n_N T_{\text{lin}}$ is one state solve, $T_{\text{lin}} = \mathcal{O}(n_c N \log N)$ one linear solve.

Method	State solves	Compute	Memory	Accuracy
Finite differences	$N+1$	$\mathcal{O}(N T_{\text{sol}})$	$\mathcal{O}(N)$	$\mathcal{O}(\sqrt{\varepsilon_{\text{mach}}})$
Forward AD (CG-ruled)	$\sim N$	$\mathcal{O}(N T_{\text{sol}})$	$\mathcal{O}(N)$	machine, at iterate
Reverse AD naive (CG-ruled)	1+ bwd	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(n_N N)$	machine, at iterate
Adjoint	1	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(N)$	exact, at u^*
Rule-based AD	1	$\mathcal{O}(T_{\text{sol}})$	$\mathcal{O}(N)$	= adjoint

2 Theoretical complexity

Let N be the number of parameters (grid points), n_N the number of Newton iterations, and n_c the number of CG iterations per linear solve. One spectral operator application costs $\mathcal{O}(N \log N)$, so one preconditioned CG solve costs $T_{\text{lin}} = \mathcal{O}(n_c N \log N)$ and one state (Newton) solve costs $T_{\text{sol}} = n_N T_{\text{lin}}$. We express each gradient cost in these units; ?? summarizes.

Two regimes. Finite differences and forward-mode AD are $\mathcal{O}(N)$ state solves: the gradient costs N times a loss evaluation, the unavoidable price of probing one input direction at a time. Reverse-mode AD, the adjoint, and rule-based AD all return the *entire* gradient at the cost of $\mathcal{O}(1)$ state solves — gradient cost comparable to loss cost, independent of the parameter count. This is the classical reverse-mode result [?, ?].

Inside the $\mathcal{O}(1)$ -solve group. The separation here is the constant and the memory. The hand adjoint adds a single linear solve on top of the state solve and stores nothing extra. Naive reverse-mode re-does the iteration backward — an extra $\approx n_N$ linear solves (so $\approx 2\times$ the adjoint compute) and a tape of the n_N Newton iterates ($\mathcal{O}(n_N N)$ memory). Rule-based AD recovers the adjoint’s single-solve cost and $\mathcal{O}(N)$ memory while staying composable, because the rule injects exactly the adjoint the hand-written method uses.

Accuracy. Forward- and reverse-mode AD differentiate the program, i.e. the n_N -th Newton iterate $u_{n_N}(V)$: they return the exact derivative of an approximate solution, differing from the true gradient by $\mathcal{O}(\text{Newton residual})$. The adjoint differentiates the converged solution through one exact linear solve. The state-solve rule injects that same adjoint, so rule-based AD matches the hand adjoint (to the linear-solve tolerance), not the iterate.

3 Protocol

We fix a well-conditioned instance — a forcing with nonzero mean, so the state stays bounded away from zero — and a single evaluation point V_0 . Before any timing, every gradient is checked against finite differences at a non-negligible regularization weight, so a method that is fast only because it is wrong is excluded. *Time* is the minimum over repeated samples (BenchmarkTools, warmed up to exclude JIT compilation, with run-time values interpolated so global lookups are not measured); *memory* is the allocation per call; *accuracy* is the relative ℓ_2 deviation from the adjoint gradient. Every time is also quoted as a multiple of a single state solve, the natural unit of ?. The measurement is repeated over $N = 5, \dots, 500$ for the scaling study. The AD tools are Enzyme, Mooncake and Zygote; all AD paths use the CG rule, and the rule-based paths add the state-solve rule.

Table 2: Relative ℓ_2 deviation from the adjoint, and the correctness-gate outcome (deviation from finite differences at $\beta = 1$).

missing table: generated/accuracy_table.tex

missing figure: figures/accuracy_vs_adjoint.pdf
(run gradient.jl)

Figure 1: Relative ℓ_2 error of each method against the adjoint (log scale).

4 Results

Accuracy. *???? report the deviation of each gradient from the adjoint reference at $N = \lceil N \rceil$. [State the FD deviation, the naive-AD deviation (the iterate gap, of order the Newton residual), and confirm that rule-based AD sits at the adjoint level rather than the iterate level.]*

Cost at fixed size. *?? gives wall time and allocations at $N = \lceil N \rceil$, with each time also as a multiple of one state solve.*

[Quote the adjoint and rule-based times in units of one solve (≈ 1 – 2), the FD and forward-AD slowdown ($\sim N$ solves), and the reverse-naive allocation relative to the adjoint, where the $\mathcal{O}(n_N N)$ tape shows up even when the time is competitive.]

Scaling in N . *???? show time and memory against N on log–log axes; ?? is the backing data. [Read off the slopes: FD and forward-AD have unit slope in time ($\mathcal{O}(N)$ solves); reverse-mode, adjoint and rule-based are flat. In memory, reverse-naive grows with N while the others stay flat. Give the crossover N beyond which the adjoint is decisively cheaper than FD.]*

5 Discussion

The measurements confirm *??*. Finite differences and forward-mode AD scale as $\mathcal{O}(N)$ state solves and are usable only as baselines. Reverse-mode AD, the adjoint, and rule-based AD all compute the full gradient at the cost of $\mathcal{O}(1)$ solves; within that group the hand adjoint is the cheapest and lightest, naive reverse-mode pays the Newton-unrolling factor in both compute ($\approx 2\times$) and memory ($\mathcal{O}(n_N N)$), and rule-based AD recovers the adjoint’s efficiency while remaining composable. For production the hand adjoint is the clear choice; rule-based AD is preferable when the gradient must chain with other differentiated code; naive differentiation is kept only as a baseline and a correctness cross-check.

Reproducibility. *[run gradient.jl to stamp version/hardware provenance here]*

Table 3: Time and allocations per gradient evaluation at fixed N .

missing table: generated/cost_{table.tex}

missing figure: figures/scaling_{time.pdf} (run gradient.jl)	missing figure: figures/scaling_{memory.pdf} (run gradient.jl)
---	---

Figure 2: Time versus N .

Figure 3: Allocations versus N .

References

- [1] S. G. Johnson, *Notes on Adjoint Methods for 18.335*, MIT, 2006 (updated 2021).
- [2] A. Levitt, *The Adjoint Trick*, 2020.

Table 4: Time and allocations across N (backing data for the scaling plots).

missing table: generated/scaling_{*t*}*able.tex*